

STEP_BY_STEP_IMPLEMENTATIO N_GUIDE

STEP-BY-STEP IMPLEMENTATION GUIDE

Universal Multi-Format Multi-Language Ebook Translation System

Date: November 25, 2025

Guide Type: Detailed Implementation Roadmap

Duration: 14 Weeks Complete Implementation

Target: 100% Completion with Full Test Coverage

OVERVIEW

This step-by-step implementation guide provides detailed instructions for completing the Universal Ebook Translator project from its current ~60% completion state to a fully production-ready system. Each phase includes specific tasks, code examples, and validation criteria.

IMPLEMENTATION PRINCIPLES

- Test-First Approach** - Write tests before implementation
 - Security-First Development** - Prioritize security in all components
 - Incremental Delivery** - Deliver working features incrementally
 - Documentation-Driven** - Document all components comprehensively
 - Production-Ready Standards** - Build for production from day one
-

PHASE 0: CRITICAL INFRASTRUCTURE STABILIZATION (WEEK 1)

DAY 1: BUILD SYSTEM FIXES (4 HOURS)

Task 1.1: Remove Conflicting Main Functions

```
# Create tools directory structure
mkdir -p tools/{debug,test,setup}

# Move conflicting files
mv debug_*.go tools/debug/
mv test_*.go tools/test/
mv setup_linting.go tools/setup/
mv simple.go tools/debug/
mv create_test_epub.go tools/test/

# Create proper Go modules for tools
cd tools/debug
go mod init digital.vasic.translator/tools/debug
go mod tidy

cd ../test
go mod init digital.vasic.translator/tools/test
go mod tidy

cd ../setup
go mod init digital.vasic.translator/tools/setup
go mod tidy
```

Task 1.2: Fix Import Cycles in Translator Package

```
// File: pkg/translator/universal_test.go
package translator_test

import (
    "testing"
    "context"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    translator "github.com/universal-ebook-translator/pkg/translator"
    "github.com/universal-ebook-translator/test/mocks"
)

func TestUniversalTranslator_Integration(t *testing.T) {
    // Setup mock translator to avoid import cycles
    mockLLM := &mocks.MockLLMProvider{}

    cfg := translator.Config{
        Provider: "mock",
    }
```

```

        Model:    "test-model",
    }

    trans, err := translator.New(cfg,
        translator.WithLLMProvider(mockLLM))
    require.NoError(t, err)

    // Test translation
    result, err := trans.Translate(context.Background(), "Hello",
        "en", "es")
    require.NoError(t, err)
    assert.NotEmpty(t, result)
}

```

Task 1.3: Fix Missing Imports

```

// File: cmd/translate-ssh/main_test.go
package main

import (
    "context"
    "fmt" // ADD THIS IMPORT
    "os"
    "testing"
    "time"
    "encoding/json" // ADD THIS IMPORT
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestSSHCommand(t *testing.T) {
    // Test implementation with proper imports
    tests := []struct {
        name      string
        args      []string
        expected  int
    }{
        {"help flag", []string{"--help"}, 0},
        {"version flag", []string{"--version"}, 0},
        {"invalid args", []string{"--invalid"}, 1},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            // Use proper JSON and fmt imports
            result := runCommand(tt.args)
            assert.Equal(t, tt.expected, result)
        })
    }
}

```

Validation Criteria

```
# Verify all tests compile
go test ./... -v 2>&1 | grep -E "(PASS|FAIL|ok)" | head -10

# Should show no compilation errors
# Expected output:
# ok      digital.vasic.translator/pkg/format      0.123s
# ok      digital.vasic.translator/pkg/logger      0.045s
# ... etc
```

DAY 2: TEST INFRASTRUCTURE SETUP (4 HOURS)

Task 2.1: Create Test Utilities Package

```
// File: test/utils/helpers.go
package utils

import (
    "context"
    "os"
    "path/filepath"
    "testing"
    "time"
    "github.com/stretchr/testify/require"
)

// TestConfig provides configuration for tests
type TestConfig struct {
    DatabaseURL string
    APIToken    string
    TestTimeout time.Duration
}

// SetupTestEnvironment creates test environment
func SetupTestEnvironment(t *testing.T) *TestConfig {
    t.Helper()

    tempDir := t.TempDir()
    dbPath := filepath.Join(tempDir, "test.db")

    return &TestConfig{
        DatabaseURL: "sqlite://" + dbPath,
        APIToken:    "test-token-" + t.Name(),
        TestTimeout: 30 * time.Second,
    }
}

// CleanupTestEnvironment cleans up test resources
func CleanupTestEnvironment(t *testing.T, config *TestConfig) {
    t.Helper()
}
```

```

    if config != nil && config.DatabaseURL != "" {
        os.Remove(config.DatabaseURL)
    }
}

// CreateTestEPUB creates a test EPUB file
func CreateTestEPUB(t *testing.T, title, content string) string {
    t.Helper()

    tempDir := t.TempDir()
    epubPath := filepath.Join(tempDir, "test.epub")

    // Create minimal EPUB structure
    mimetype := "application/epub+zip"

    // Create container.xml
    containerXML := `<?xml version="1.0"?>
<container version="1.0"
    xmlns="urn:oasis:names:tc:opendocument:xmlns:container">
    <rootfiles>
        <rootfile full-path="OEBPS/content.opf" media-
            type="application/oebps-package+xml"/>
    </rootfiles>
</container>`

    // Create content.opf
    contentOPF := fmt.Sprintf(`<?xml version="1.0"?>
<package version="3.0" xmlns="http://www.idpf.org/2007/opf">
    <metadata>
        <dc:title xmlns:dc="http://purl.org/dc/elements/1.1/">%s</
            dc:title>
    </metadata>
    <manifest>
        <item id="chapter1" href="chapter1.xhtml" media-
            type="application/xhtml+xml"/>
    </manifest>
    <spine>
        <itemref idref="chapter1"/>
    </spine>
</package>`, title)

    // Create chapter1.xhtml
    chapterXHTML := fmt.Sprintf(`<?xml version="1.0"?>
<html xmlns="http://www.w3.org/1999/xhtml">
<head><title>%s</title></head>
<body><p>%s</p></body>
</html>`, title, content)

    // Create EPUB file (simplified - real implementation would
    use zip)
    // For tests, we'll create a placeholder file
    require.NoError(t, os.WriteFile(epubPath, []byte("test-epub-
        content"), 0644))

```

```

    return epubPath
}

// CreateTestContext creates a test context with timeout
func CreateTestContext(t *testing.T) context.Context {
    t.Helper()
    ctx, cancel := context.WithTimeout(context.Background(),
        30*time.Second)
    t.Cleanup(cancel)
    return ctx
}

```

Task 2.2: Create Mock Implementations

```

// File: test/mocks/llm_provider.go
package mocks

import (
    "context"
    "github.com/stretchr/testify/mock"
)

// MockLLMProvider implements LLM provider interface for testing
type MockLLMProvider struct {
    mock.Mock
}

func (m *MockLLMProvider) Translate(ctx context.Context, text,
    from, to string) (string, error) {
    args := m.Called(ctx, text, from, to)
    return args.String(0), args.Error(1)
}

func (m *MockLLMProvider) GetProvider() string {
    args := m.Called()
    return args.String(0)
}

func (m *MockLLMProvider) IsAvailable(ctx context.Context) bool {
    args := m.Called(ctx)
    return args.Bool(0)
}

// MockDatabase implements database interface for testing
type MockDatabase struct {
    mock.Mock
}

func (m *MockDatabase) SaveTranslation(source, target,
    sourceLang, targetLang string) error {
    args := m.Called(source, target, sourceLang, targetLang)
    return args.Error(0)
}

```

```

}

func (m *MockDatabase) GetTranslation(source, sourceLang,
    targetLang string) (string, error) {
    args := m.Called(source, sourceLang, targetLang)
    return args.String(0), args.Error(1)
}

func (m *MockDatabase) Close() error {
    args := m.Called()
    return args.Error(0)
}

// MockSecurityProvider implements security interface for testing
type MockSecurityProvider struct {
    mock.Mock
}

func (m *MockSecurityProvider) Authenticate(token string) (bool,
    error) {
    args := m.Called(token)
    return args.Bool(0), args.Error(1)
}

func (m *MockSecurityProvider) Authorize(user, resource string)
    (bool, error) {
    args := m.Called(user, resource)
    return args.Bool(0), args.Error(1)
}

```

Task 2.3: Set Up Test Data

```

# File: test/fixtures/create_test_data.sh
#!/bin/bash

# Create test data directory
mkdir -p test/fixtures/ebooks
mkdir -p test/fixtures/translations
mkdir -p test/fixtures/configs

# Create test ebooks
cat > test/fixtures/ebooks/sample.txt << 'EOF'
This is a sample English text for testing translation
    functionality.
It contains multiple sentences and should be suitable for
    translation testing.
EOF

cat > test/fixtures/ebooks/sample.html << 'EOF'
<!DOCTYPE html>
<html>
<head><title>Sample Document</title></head>
<body>

```

```
<h1>Test Document</h1>
<p>This is a test HTML document for translation.</p>
<p>It contains headings and paragraphs.</p>
</body>
</html>
EOF
```

```
cat > test/fixtures/ebooks/sample.fb2 << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<FictionBook xmlns="http://www.gribuser.ru/xml/fictionbook/2.0">
  <description>
    <title-info>
      <book-title>Test Book</book-title>
      <lang>en</lang>
    </title-info>
  </description>
  <body>
    <section>
      <title><p>Test Chapter</p></title>
      <p>This is a test FB2 document.</p>
      <p>It contains section titles and paragraphs.</p>
    </section>
  </body>
</FictionBook>
EOF
```

```
# Create test configurations
```

```
cat > test/fixtures/configs/test_config.json << 'EOF'
{
  "provider": "mock",
  "model": "test-model",
  "temperature": 0.3,
  "max_tokens": 1000,
  "target_language": "es",
  "output_format": "epub"
}
EOF
```

```
cat > test/fixtures/configs/performance_test.json << 'EOF'
{
  "provider": "mock",
  "model": "performance-test",
  "temperature": 0.1,
  "max_tokens": 2000,
  "target_language": "fr",
  "output_format": "txt"
}
EOF
```

```
echo "Test data created successfully"
```

Validation Criteria

```
# Verify test infrastructure
go test ./test/utils -v
go test ./test/mocks -v

# Verify test data creation
ls -la test/fixtures/
# Should show:
# ebooks/      translations/      configs/
```

DAY 3: COVERAGE ANALYSIS & PRIORITIZATION (4 HOURS)

Task 3.1: Generate Comprehensive Coverage Report

```
#!/bin/bash
# File: scripts/generate_coverage_report.sh

echo "Generating comprehensive coverage report..."

# Clean previous coverage files
rm -f coverage*.out coverage*.html

# Generate coverage for all packages
go test ./... -coverprofile=coverage-full.out -v 2>&1 | tee
    coverage-test.log

# Generate HTML report
go tool cover -html=coverage-full.out -o coverage-full.html

# Extract coverage by package
echo "Coverage by package:"
go tool cover -func=coverage-full.out | grep "^pkg/" | sort -k3 -n

# Identify low coverage packages
echo ""
echo "Low coverage packages (<80%):"
go tool cover -func=coverage-full.out | awk '$3 < "80.0%" && $1
    ~ /^pkg/ {print $1, $3}'

# Identify packages with no tests
echo ""
echo "Packages with no tests:"
find pkg/ -name "*.go" -not -name "*_test.go" | while read file;
do
    pkg=$(dirname "$file")
    if ! go test ".$pkg" -v 2>&1 | grep -q "no test files"; then
        if ! ls "$pkg"/*_test.go >/dev/null 2>&1; then
            echo "$pkg"
        fi
    fi
fi
```

```
done | sort -u
```

```
echo "Coverage report generated: coverage-full.html"
```

Task 3.2: Create Priority Matrix

```
// File: test/coverage/priority_matrix.go
```

```
package coverage
```

```
import (  
    "fmt"  
    "sort"  
)
```

```
type PriorityLevel int
```

```
const (  
    Critical PriorityLevel = iota  
    High  
    Medium  
    Low  
)
```

```
type Component struct {  
    Package      string  
    Current      float64  
    Target       float64  
    Priority      PriorityLevel  
    Security      bool  
    CoreFunction bool  
}
```

```
func GeneratePriorityMatrix() []Component {  
    return []Component{  
        // Critical Security Components  
        {"pkg/security/", 20.0, 95.0, Critical, true, true},  
        {"pkg/distributed/", 60.0, 90.0, Critical, true, true},  
  
        // Core Business Logic  
        {"pkg/translator/", 33.0, 90.0, Critical, false, true},  
        {"pkg/models/", 25.0, 90.0, High, false, true},  
        {"pkg/api/", 33.0, 85.0, High, false, true},  
  
        // Supporting Components  
        {"pkg/markdown/", 43.0, 85.0, Medium, false, false},  
        {"pkg/preparation/", 25.0, 85.0, Medium, false, false},  
        {"pkg/report/", 0.0, 80.0, Medium, false, false},  
  
        // Tools and Utilities  
        {"cmd/", 10.0, 70.0, Low, false, false},  
    }  
}
```

```

func (c Component) GetScore() float64 {
    score := c.Target - c.Current

    if c.Priority == Critical {
        score *= 3.0
    } else if c.Priority == High {
        score *= 2.0
    }

    if c.Security {
        score *= 1.5
    }

    if c.CoreFunction {
        score *= 1.2
    }

    return score
}

func PrintPriorityMatrix() {
    components := GeneratePriorityMatrix()

    // Sort by priority score (highest first)
    sort.Slice(components, func(i, j int) bool {
        return components[i].GetScore() > components[j].GetScore()
    })

    fmt.Printf("%-20s %-10s %-10s %-10s %-10s %-15s\n",
        "Package", "Current", "Target", "Priority", "Security",
        "Score")
    fmt.Println(strings.Repeat("-", 80))

    for _, comp := range components {
        fmt.Printf("%-20s %-9.1f%% %-9.1f%% %-10v %-10v
%-14.1f\n",
            comp.Package, comp.Current, comp.Target,
            comp.Priority, comp.Security, comp.GetScore())
    }
}

```

Validation Criteria

```

# Generate coverage report
./scripts/generate_coverage_report.sh

# Should show:
# 1. Complete coverage analysis
# 2. Low coverage identification
# 3. Priority matrix
# 4. HTML report for visualization

```

PHASE 1: CRITICAL TEST COVERAGE IMPLEMENTATION (WEEKS 2-4)

WEEK 2: SECURITY & DISTRIBUTED SYSTEM TESTS

DAY 4-5: SECURITY PACKAGE TESTS

Task 4.1: User Authentication Tests

// File: pkg/security/user_auth_test.go

package security

import (

 "context"

 "testing"

 "time"

 "github.com/stretchr/testify/assert"

 "github.com/stretchr/testify/require"

 "golang.org/x/crypto/bcrypt"

)

func TestJWTAuthentication(t *testing.T) {

 auth := NewJWTAuthenticator("test-secret-key")

 t.Run("ValidTokenGeneration", **func**(t *testing.T) {

 userID := "user123"

 token, err := auth.GenerateToken(userID, 24*time.Hour)

 require.NoError(t, err)

 assert.NotEmpty(t, token)

// Verify token structure

 parts := strings.Split(token, ".")

 assert.Len(t, parts, 3) *// header.payload.signature*

 })

 t.Run("ValidTokenValidation", **func**(t *testing.T) {

 userID := "user123"

 token, err := auth.GenerateToken(userID, 24*time.Hour)

 require.NoError(t, err)

 parsedUserID, err := auth.ValidateToken(token)

 require.NoError(t, err)

 assert.Equal(t, userID, parsedUserID)

 })

 t.Run("InvalidTokenRejection", **func**(t *testing.T) {

 invalidTokens := []string{

 "", *// Empty*

 "invalid.token", *// Malformed*

 "invalid.invalid.invalid", *// Invalid signature*

```
"eyJhbGciOiJIub250IiwidHlwIjoiSldUIIn0.eyJzdWIiOiIxMjMONTYzODkwIiwibmFtZSI6Im9ybmVudCJ9.CXQwNjUxMTk2NDc3OTBhbnRlLnNo signature
}

    for _, token := range invalidTokens {
        _, err := auth.ValidateToken(token)
        assert.Error(t, err, "Token %q should be invalid",
            token)
    }
})

t.Run("ExpiredToken", func(t *testing.T) {
    userID := "user123"
    token, err := auth.GenerateToken(userID,
        1*time.Millisecond)
    require.NoError(t, err)

    // Wait for token to expire
    time.Sleep(10 * time.Millisecond)

    _, err = auth.ValidateToken(token)
    assert.Error(t, err)
    assert.Contains(t, err.Error(), "expired")
})
}
```

```
func TestPasswordHashing(t *testing.T) {
    t.Run("PasswordHashing", func(t *testing.T) {
        password := "secure-password-123"

        hash, err := HashPassword(password)
        require.NoError(t, err)
        assert.NotEmpty(t, hash)
        assert.NotEqual(t, password, hash)

        // Verify hash starts with bcrypt prefix
        assert.True(t, strings.HasPrefix(hash, "$2"))
    })
}

t.Run("PasswordVerification", func(t *testing.T) {
    password := "secure-password-123"
    hash, err := HashPassword(password)
    require.NoError(t, err)

    // Correct password should validate
    err = VerifyPassword(hash, password)
    assert.NoError(t, err)

    // Wrong password should not validate
    err = VerifyPassword(hash, "wrong-password")
    assert.Error(t, err)
    assert.Equal(t, bcrypt.ErrMismatchedHashAndPassword, err)
```

```

    })

    t.Run("TimingAttackResistance", func(t *testing.T) {
        password := "secure-password-123"
        hash, err := HashPassword(password)
        require.NoError(t, err)

        // Measure timing for correct password
        start := time.Now()
        VerifyPassword(hash, password)
        correctDuration := time.Since(start)

        // Measure timing for wrong password
        start = time.Now()
        VerifyPassword(hash, "wrong-password")
        wrongDuration := time.Since(start)

        // Duration should be similar (within reasonable
        // tolerance)
        ratio := float64(wrongDuration) / float64(correctDuration)
        assert.Greater(t, ratio, 0.5) // Not more than 2x
        // difference
        assert.Less(t, ratio, 2.0)    // Not less than 0.5x
        // difference
    })
}

func TestRateLimiting(t *testing.T) {
    limiter := NewRateLimiter(RateLimiterConfig{
        MaxRequests: 5,
        Window:      time.Minute,
    })

    t.Run("RateLimitWithinThreshold", func(t *testing.T) {
        userID := "user123"

        // Should allow requests within limit
        for i := 0; i < 5; i++ {
            allowed := limiter.Allow(userID)
            assert.True(t, allowed, "Request %d should be
            allowed", i+1)
        }
    })

    t.Run("RateLimitExceedsThreshold", func(t *testing.T) {
        userID := "user456"

        // Exceed limit
        for i := 0; i < 10; i++ {
            limiter.Allow(userID)
        }

        // Next request should be denied
    })
}

```

```

        allowed := limiter.Allow(userID)
        assert.False(t, allowed, "Request exceeding limit should
        be denied")
    })

    t.Run("RateLimitReset", func(t *testing.T) {
        userID := "user789"

        // Fill up the rate limit
        for i := 0; i < 5; i++ {
            limiter.Allow(userID)
        }

        // Should be denied
        assert.False(t, limiter.Allow(userID))

        // Simulate time window passing (in real implementation,
        this would be time-based)
        limiter.ResetWindow(userID)

        // Should be allowed again
        assert.True(t, limiter.Allow(userID))
    })
}

func TestInputSanitization(t *testing.T) {
    sanitizer := NewInputSanitizer()

    t.Run("SQLInjectionPrevention", func(t *testing.T) {
        maliciousInputs := []string{
            "'; DROP TABLE users; --",
            "'; DELETE FROM translations; --",
            "' OR '1'='1",

            "'1'; UPDATE users SET password='hacked' WHERE '1'='1';
            --",
        }

        for _, input := range maliciousInputs {
            sanitized := sanitizer.Sanitize(input)
            assert.NotContains(t, sanitized, "DROP",
            "Input should not contain DROP keyword")
            assert.NotContains(t, sanitized, "DELETE", "Input
            should not contain DELETE keyword")
            assert.NotContains(t, sanitized, "UPDATE", "Input
            should not contain UPDATE keyword")
            assert.NotContains(t, sanitized, "--", "Input should
            not contain SQL comment")
        }
    })

    t.Run("XSSPrevention", func(t *testing.T) {
        maliciousInputs := []string{
            "<script>alert('xss')</script>",

```

```

        "<img src=x onerror=alert('xss')>",
        "<svg onload=alert('xss')>",
        "javascript:alert('xss')",
    }

    for _, input := range maliciousInputs {
        sanitized := sanitizer.Sanitize(input)
        assert.NotContains(t, sanitized, "<script>", "Input
        should not contain script tags")
        assert.NotContains(t, sanitized, "onerror=", "Input
        should not contain event handlers")
        assert.NotContains(t, sanitized, "onload=", "Input
        should not contain event handlers")
        assert.NotContains(t, sanitized, "javascript:",
        "Input should not contain javascript protocol")
    }
})

t.Run("PathTraversalPrevention", func(t *testing.T) {
    maliciousInputs := []string{
        "../../../etc/passwd",
        "..\\..\\..\\windows\\system32\\config\\sam",
        "....//....//....//etc/passwd",
        "%2e%2e%2f%2e%2f%2e%2fetc%2fpasswd",
    }

    for _, input := range maliciousInputs {
        sanitized := sanitizer.Sanitize(input)
        assert.NotContains(t, sanitized, "..", "Input should
        not contain directory traversal")
        assert.NotContains(t, sanitized, "\\ ", "Input should
        not contain backslashes")
    }
})
}

```

Task 4.2: Security Integration Tests

// File: pkg/security/security_integration_test.go

```

package security_test

import (
    "context"
    "net/http"
    "net/http/httptest"
    "testing"
    "time"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "github.com/universal-ebook-translator/pkg/security"
    "github.com/universal-ebook-translator/pkg/api"
)

```

```

func TestSecurityIntegration(t *testing.T) {
    // Setup test server with security middleware
    auth := security.NewJWTAuthenticator("test-secret")
    limiter := security.NewRateLimiter(security.RateLimiterConfig{
        MaxRequests: 10,
        Window:      time.Minute,
    })

    server :=
        httptest.NewServer(api.SetupSecurityMiddleware(http.HandlerFunc(func(w
            http.ResponseWriter, r *http.Request) {
                w.WriteHeader(http.StatusOK)
                w.Write([]byte("success"))
            })), auth, limiter))
    defer server.Close()

    t.Run("ValidAuthentication", func(t *testing.T) {
        token, err := auth.GenerateToken("testuser", time.Hour)
        require.NoError(t, err)

        req, err := http.NewRequest("GET", server.URL, nil)
        require.NoError(t, err)
        req.Header.Set("Authorization", "Bearer "+token)

        resp, err := http.DefaultClient.Do(req)
        require.NoError(t, err)
        defer resp.Body.Close()

        assert.Equal(t, http.StatusOK, resp.StatusCode)
    })

    t.Run("InvalidAuthentication", func(t *testing.T) {
        req, err := http.NewRequest("GET", server.URL, nil)
        require.NoError(t, err)
        req.Header.Set("Authorization", "Bearer invalid-token")

        resp, err := http.DefaultClient.Do(req)
        require.NoError(t, err)
        defer resp.Body.Close()

        assert.Equal(t, http.StatusUnauthorized, resp.StatusCode)
    })

    t.Run("RateLimitingIntegration", func(t *testing.T) {
        token, err := auth.GenerateToken("testuser2", time.Hour)
        require.NoError(t, err)

        // Make requests up to the limit
        for i := 0; i < 10; i++ {
            req, err := http.NewRequest("GET", server.URL, nil)
            require.NoError(t, err)
            req.Header.Set("Authorization", "Bearer "+token)

```

```

        resp, err := http.DefaultClient.Do(req)
        require.NoError(t, err)
        resp.Body.Close()

        if i < 9 {
            assert.Equal(t, http.StatusOK, resp.StatusCode,
                "Request %d should succeed", i+1)
        }
    }

    // Next request should be rate limited
    req, err := http.NewRequest("GET", server.URL, nil)
    require.NoError(t, err)
    req.Header.Set("Authorization", "Bearer "+token)

    resp, err := http.DefaultClient.Do(req)
    require.NoError(t, err)
    defer resp.Body.Close()

    assert.Equal(t, http.StatusTooManyRequests,
        resp.StatusCode)
    })
}

```

Validation Criteria

```

# Run security tests
go test ./pkg/security -v -race -cover

# Expected results:
# ok      digital.vasic.translator/pkg/security    1.234s
# coverage: 95.2% of statements

# Run integration tests
go test ./test/integration/security_test.go -v

# Expected results:
# ok      security integration tests             0.456s

```

CONTINUING GUIDE STRUCTURE

Due to space constraints, I've outlined the beginning of this comprehensive implementation guide. The complete guide would continue with:

WEEK 2-3: Remaining Critical Tests

- Distributed system tests (SSH, connection pooling)
- Core translator tests (LLM providers, translation logic)
- API handler tests (all endpoints, authentication, rate limiting)

WEEK 4: Integration & Performance Tests

- Database integration tests
- External service integration tests
- Performance benchmarks and stress tests

WEEK 5-6: Comprehensive Test Framework

- Security test suite completion
- User acceptance tests
- Test automation and CI/CD integration

WEEK 7-8: Documentation Completion

- Architecture documentation with diagrams
- API documentation with interactive examples
- User manuals and troubleshooting guides

WEEK 9-10: Website Development

- Interactive demos and API playground
- Video course integration
- Content management system setup

WEEK 11-12: Video Course Production

- 19 professional videos across 3 course series
- Production quality standards and review process
- Supplementary materials and transcripts

WEEK 13-14: Production Readiness

- Monitoring and observability infrastructure
 - Security hardening and audit
 - Production deployment and CI/CD
-

DAILY IMPLEMENTATION SCHEDULE

Daily Structure (Monday-Friday, 8 hours/day)

- **2 hours:** Code implementation and testing
- **1 hour:** Documentation writing
- **1 hour:** Code review and quality assurance
- **2 hours:** Additional development/implementation
- **1 hour:** Integration testing
- **1 hour:** Daily progress review and planning

Weekly Milestones

Each week ends with a comprehensive review and delivery of specific components as outlined in the validation criteria.

SUCCESS METRICS

Weekly Deliverables

- All code passes tests with 95%+ coverage
- Documentation is complete and accurate
- Components are production-ready
- No broken or disabled modules remain

Final Success Criteria

-  100% test coverage across all packages
 -  Complete documentation suite
 -  19 professional video courses
 -  Full-featured website with interactive demos
 -  Production-ready monitoring and security
-

This detailed step-by-step guide provides the complete roadmap to transform the Universal Ebook Translator from its current state to a fully production-ready system with comprehensive testing, documentation, and user resources.

Guide generated November 25, 2025 Next update: Weekly progress reports during implementation